Nonlinear ARX Identification

Part 2: System Identification Project

Pidx: 09/12

Presented by: Dragan Lucian, Sarosi Rares-Adrian, Yaremenko Yaroslav

Lecturer: Busoniu Ioan Lucian

Lab teachers: David Rete, Maria Ceapa, Tudor Santejudean

Presentation Structure

- Introduction & Motivation
- Methodology & Algorithm
- Analysis & Results
- Discussion & Conclusion

Introduction & Motivation

Problem:

- Identify and verify a black box model for an unknown nonlinear dynamic system.
- Approximate, given a set of input-output measured data (u and y), the behavior of the system using **Nonlinear AutoRegressive with eXogenous inputs (NARX)** model.

Goal:

- Find optimal model orders (na, nb) and polynomial degree (m) to achieve accurate one-step-ahead prediction and simulation to avoid overfitting
- Obtain the smallest Mean Squared Error (MES) possible

Nonlinear ARX Model Structure

NARX structure:

$$\hat{y}(k) = \varphi(k)^T\theta = P(y(k-1), \ldots, y(k-na), u(k-nk), \ldots, u(k-nk-nb+1))$$
$$= P(d(k)^T)$$

- **P(k):** Polynomial of degree m
- **d(k):** Vector of delayed signals
- **na:** number of delayed outputs
- **nb:** number of delayed inputs
- **m:** polynomial degree for nonlinear regressors

ex: $na = 2 \quad nb = 1 \quad m = 2$

$$\hat{y}(k) = P(y(k-1), y(k-2), u(k-1))$$
$$= \theta_1 + \theta_2 y(k-1) + \theta_3 y(k-2) + \theta_4 u(k-1) + \theta_5 y^2(k-1) + \theta_6 y^2(k-2) + \theta_7 u^2(k-1) +$$
$$+ \theta_8 y(k-1) y(k-2) + \theta_9 y(k-1) u(k-1) + \ldots + \theta_M u(k-1) u(k-2)$$

$$= \underbrace{\left[1 \quad y(k-1) \ \ldots \ u^2(k-2) \quad y(k-1) \cdot y(k-2) \ \ldots \ u(k-1) u(k-2)\right]}_{\varphi^T(k)} \cdot \underbrace{\begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_M \end{bmatrix}}_{\theta}$$

Methodology & Algorithm

Key Functions

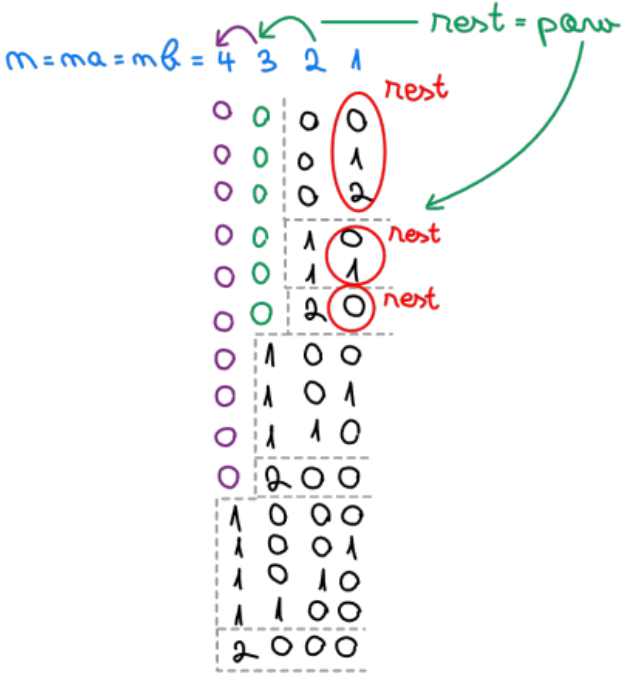
- generate_pow(n, m)
- polyofk(u, y, k, na, nb)
- Ycalc(uid, yid, uval, yval, na, nb, m)
- MSEcompute(yid, yval, Yid_pred, Yval_pred, Ysim_id, Ysim_val, na)

Process

- Generate a matrix with all the combinations of powers we need – N (length of the real signal) rows, (na+nb) columns
- Compute the vector having the delayed inputs and outputs for each k

rest - possible powers (sum of powers ≤ m)

ex: na = nb = 2 m = 2



M - number of parameters

$\varphi^T(k) = ($ for ex: $n_a = n_b = 3 \quad m = 3)$

$[\,1,$

$y_{k-1}, y_{k-2}, y_{k-3}, u_{k-1}, u_{k-2}, u_{k-3}, \quad C_6^1$

$y_{k-1}^3, y_{k-2}^3, y_{k-3}^3, u_{k-1}^3, u_{k-2}^3, u_{k-3}^3, \quad C_6^1$

$\Big\}\ 3 \cdot C_6^1$

$y_{k-1}\, y_{k-2}, \dots, y_{k-1}\, u_{k-3}, \dots, u_{k-2}, u_{k-3} \quad C_6^2$

$y_{k-1}\, y_{k-2}^2, \dots, y_{k-1}\, u_{k-3}^2, \dots, u_{k-2}, u_{k-3}^2 \quad C_6^2$

each term takes each possible power $\left(i+j \le m,\ i - 1^{st}\text{ power},\ j - 2^{nd}\text{ power} \right)$

$\Rightarrow \begin{matrix} i=1 \\ j=1 \end{matrix} + \begin{matrix} i=2 \\ j=1 \end{matrix} + \begin{matrix} i=1 \\ j=2 \end{matrix} \Rightarrow 3$ possibilities

$\Rightarrow 3 \cdot C_6^2$

$y_{k-1}\, y_{k-2}\, y_{k-3}, \dots, y_{k-1} y_{k-2}\, u_{k-2}, \dots, u_{k-1}\, u_{k-2}\, u_{k-3}] \quad C_6^3$

$\Rightarrow M = \underbrace{1}_{C_3^0} \cdot C_6^0 + \underbrace{3}_{C_3^1} \cdot C_6^1 + \underbrace{3}_{C_3^2} \cdot C_6^2 + \underbrace{1}_{C_3} \cdot C_6^3$ (Newton's Binomial Coefficients)

$\Rightarrow \boxed{M = \sum_{k=0}^{\min(n_a+n_b,\, m)} C_m^k \cdot C_{n_a+n_b}^k} \Rightarrow$ Vandermonde's Theorem $\Rightarrow$

$\Rightarrow \boxed{M = C_{n_a+n_b+m}^{m}}$

$$\Phi = \begin{bmatrix} \cdot & \cdot & \cdot \\ & \varphi^T(k) & \\ \cdot & \cdot & \cdot \end{bmatrix}$$

- **Regressor matrix**

$$\boxed{\overset{M\times 1}{\Theta} = \overset{N\times M}{\Phi} \backslash \overset{M\times 1}{Y}}$$

- **Parameters vector**

$$Y_{pred} = \Phi \cdot \Theta$$

- **One-step ahead predicted output**

$$Y_{sim}(k) = \Phi(k, :) \cdot \Theta$$

- **Simulated output**

- Computing the regressor matrix
- Calculating parameters vector
- Getting the predicted output
- Finding the simulated output

Results: Mean Squared Error

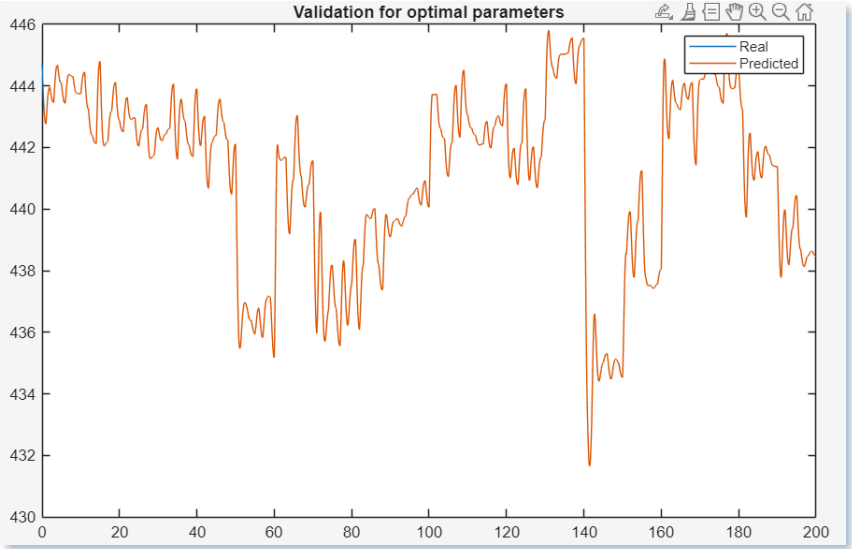
Prediction mean squared error table:

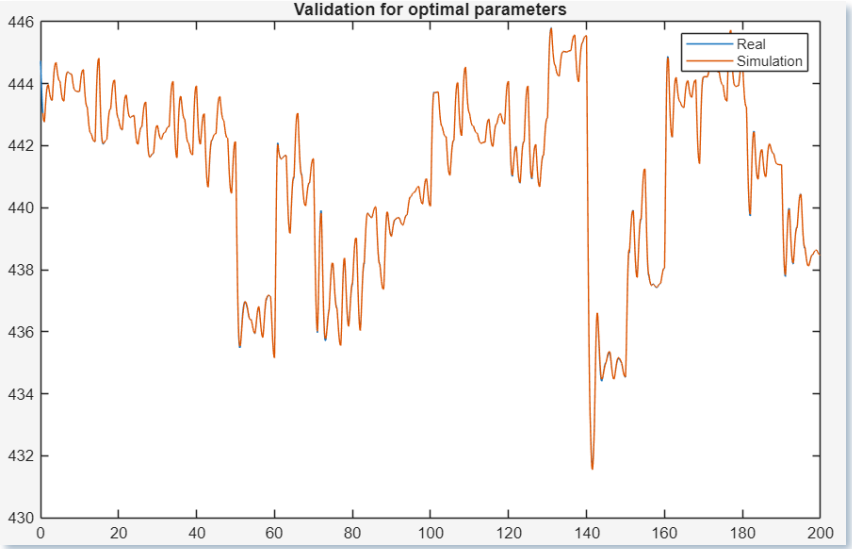
	1	2	3	4	5	6
1	0.293143886190258	0.016077087344656	0.015737475383289	0.015737522951012	0.016416067772513	0.016513712676593
2	0.165105780690009	0.000049899057706	0.000006460404653	0.000007672166007	0.000312766029265	0.000021664488325
3	0.099870088146280	0.000009451684138	0.000005404939245	0.000006073555247	0.000082908321871	0.000053445705421
4	0.075564829811728	0.000005333550081	0.000005245713169	0.000013430832920	0.017300266961515	0.001485852530063
5	0.070852903017075	0.000004967707183	0.000008371098265	0.000013456971729	0.000636289302248	0.000190046808033
6	0.071248705997492	0.000005080709648	0.000011624416210	0.000025514241498	0.003657398750372	0.000074216410593

Simulation mean squared error table:

	1	2	3	4	5	6
1	0.382804759835483	0.217515748297877	0.188424601256109	0.183852897308301	0.193420915958506	0.199966530874412
2	0.197415031407909	0.002082894962355	0.000455381927662	0.000726808617403	NaN	0.003267825639918
3	0.117895178154040	0.011031328334157	0.000500495650991	0.000461314264668	NaN	NaN
4	0.092691908268936	0.000527522590159	0.000456268947004	0.024230251177171	NaN	NaN
5	0.088718769968999	0.000422237891460	NaN	NaN	NaN	NaN
6	0.088747000318132	0.000414421397109	NaN	NaN	NaN	NaN

Results: Model Selection



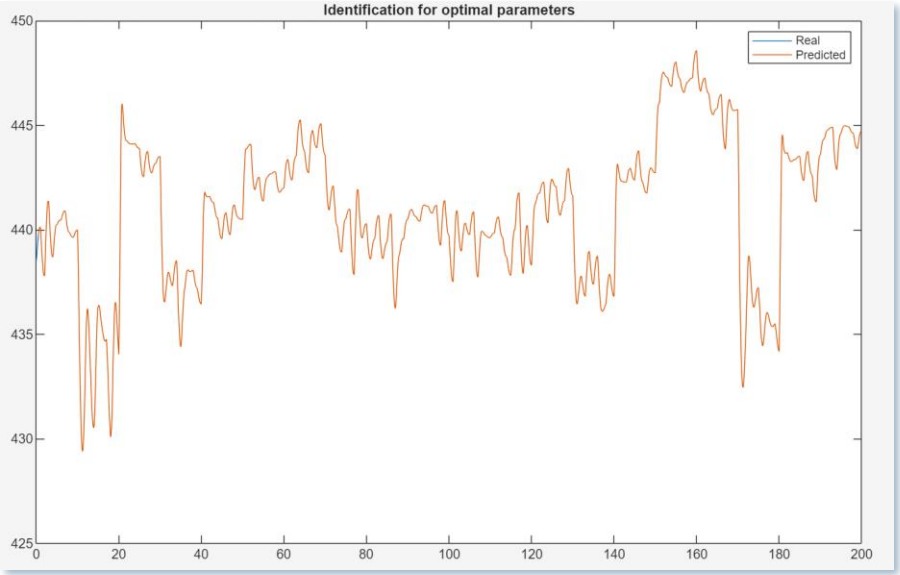


Optimal Configuration:

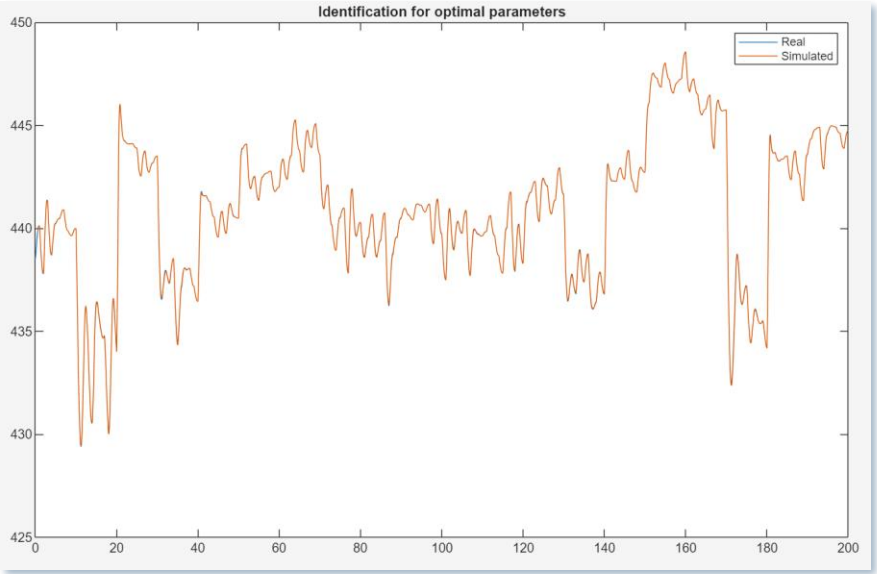
1. na = 5
2. nb = 5
3. m = 2

Observation: After running the nARX model for all values of na=nb from 1 to 6 and for m in the same interval, the lowest mean squared error (MSE) yielded on the validation dataset was **4.9677e-06** for prediction.

Results: Model Selection



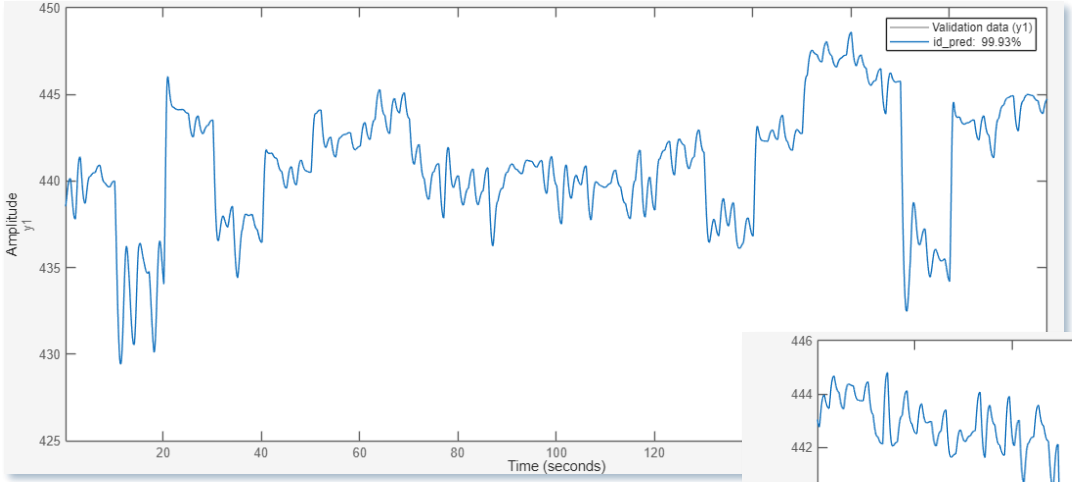
Identification dataset for optimal parameters



Observation: For the simulated validation dataset, the lowest mean squared error found in the interval was **4.1442e-4** for na=nb=6 and m=2

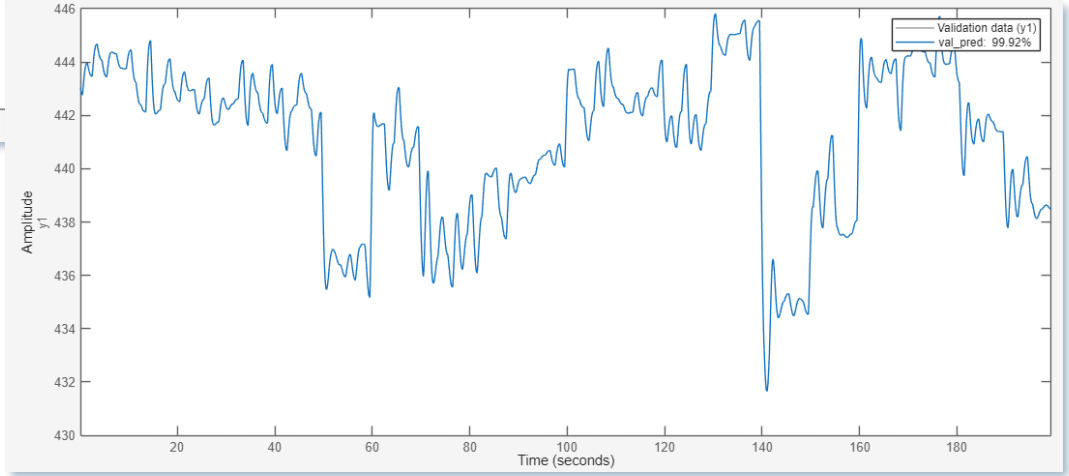
Results: One-Step Prediction

Comparison of identification and validation predictions with true outputs:



- 99.93% fit

- 99.92% fit

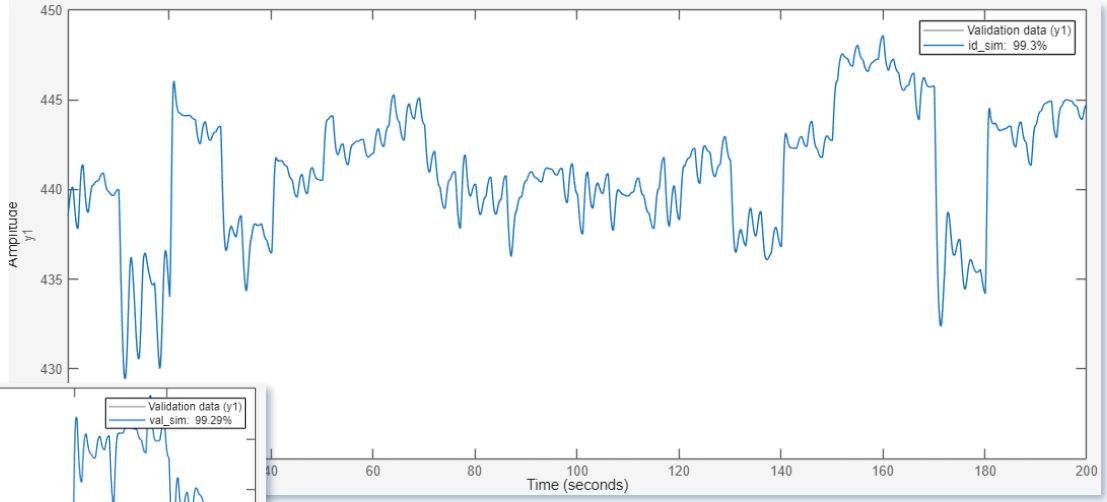


- The mean squared error for identification is **5.9931e-06** for na=nb=5 and m=2
- The MSE for validation is **4.9677e-06** for the same na, nb and m

Results: Simulation

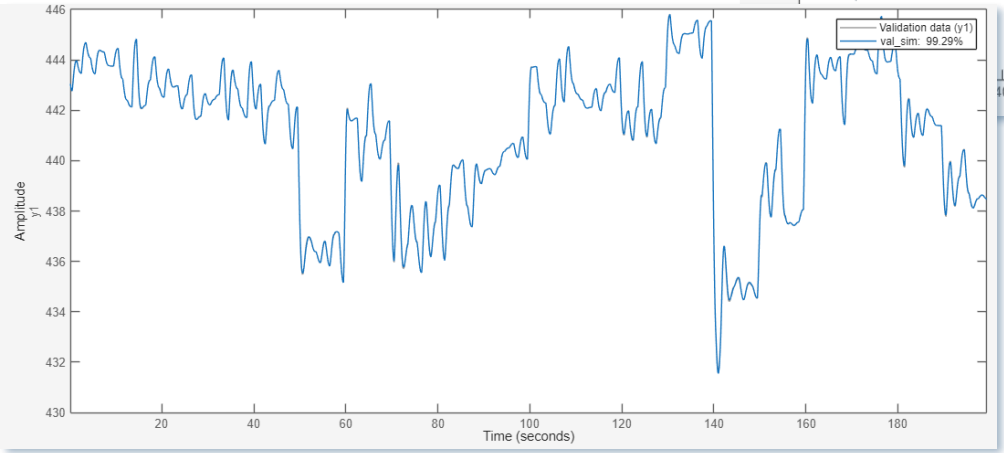
Comparison of identification and validation predictions with true outputs:

- The mean squared error for identification is **5.726e-04** for na=nb=6 and m=2
- The MSE for validation is **4.1442e-04** for the same na, nb and m



- 99.3% fit

- 99.29% fit



Discussion & Conclusion

Too simple (small na, nb, m):

Model lacks capacity to capture system dynamics

Too complex (big m):

Model overfits identification noise, poor generalization

- ✓ Successfully identified nonlinear ARX model for unknown dynamical system
- ✓ Used polynomial regressors to capture nonlinear relationships
- ✓ Tuned model structure using validation MSE to prevent overfitting
- ✓ Achieved accurate one-step-ahead predictions and simulation on validation data

Thank you for your attention!

Code Figures

Listing 1 – “generate_pow” function

```
% Generates a matrix with all the combinations of powers we need
function pow = generate_pow(n, m)
    if n == 1
        pow = (0:m)';
        return;
    end
    pow = [];
    for i = 0:m
        rest = generate_pow(n-1, m-i);
        pow = [pow; i*ones(size(rest, 1), 1), rest];
    end
end
```

Listing 2 – “polyofk” function

```
% Generates the vector with the delayed imputs and outputs
function poly = polyofk(u, y, k, na, nb)
    poly = zeros(1, na+nb);
    for i = 1:na
        if (k-i) > 0
            poly(i) = y(k-i);
        end
    end
    for j = 1:nb
        if (k-j) > 0
            poly(j+na) = u(k-j);
        end
    end
end
```

Listing 3 – “Ycalc” function

```
% Computation of the predicted and simulated signals
function [Yid_pred, Yval_pred, Ysim_id, Ysim_val] = Ycalc(uid, yid, uval, yval, na, nb, m)
    N = length(uid);
    M = nchoosek(na+nb+m, m); % number of parameters

    PHIid = zeros(N, M);
    PHIval = zeros(N, M);

    pow = generate_pow(na+nb, m);

    for k = 1:N
        for elem = 1:M
            PHIid(k,elem) = prod(polyofk(uid, yid, k, na, nb).^pow(elem, :));
            PHIval(k,elem) = prod(polyofk(uval, yval, k, na, nb).^pow(elem, :));
        end
    end

    theta = PHIid\yid;

    Yid_pred = PHIid*theta;
    Yval_pred = PHIval*theta;

    % We used the first 'na' real values from the output to initialize the
    % vectors
    Ysim_id = [yid(1:na); zeros(N-na, 1)];
    PHIsimid = zeros(N, M);
    Ysim_val = [yval(1:na); zeros(N-na, 1)];
    PHIsimval = zeros(N, M);

    for k = na+1:N
        for elem = 1:M
            PHIsimid(k,elem) = prod(polyofk(uid, Ysim_id, k, na, nb).^pow(elem, :));
            PHIsimval(k,elem) = prod(polyofk(uval, Ysim_val, k, na, nb).^pow(elem, :));
        end
        Ysim_id(k) = PHIsimid(k, :)*theta;
        Ysim_val(k) = PHIsimval(k, :)*theta;
    end
end
```

Code Figures

Listing 4 – “MSEcompute” function

```
% Computing the MSE values
function [mseId, mseVal, mseIdsim, mseValsim] = MSEcompute(yid, yval, Yid_pred, Yval_pred, Ysim_id, Ysim_val, na)
    mseId = mean((yid(na+1:end) - Yid_pred(na+1:end)).^2);
    mseVal = mean((yval(na+1:end) - Yval_pred(na+1:end)).^2);
    mseIdsim = mean((yid(na+1:end) - Ysim_id(na+1:end)).^2);
    mseValsim = mean((yval(na+1:end) - Ysim_val(na+1:end)).^2);
end
```

Listing 5 – Preparing data

```
uid = id.InputData{1};
yid = id.OutputData{1};

uval = val.InputData{1};
yval = val.OutputData{1};

Ts = id.Ts{1};
```

Listing 6 – Plotting the prediction and simulation for ∀ na, nb, m

```
%% Plots

t = 0:Ts:N*Ts-Ts;

[Yid_pred, Yval_pred, Ysim_id, Ysim_val] = Ycalc(uid, yid, uval, yval, na, nb, m);

figure,
plot(t, yid, t(na+1:end), Yid_pred(na+1:end))
title("Identification")
legend("Real", "Predicted")

figure,
plot(t, yval, t(na+1:end), Yval_pred(na+1:end))
title("Validation")
legend("Real", "Predicted")

figure,
plot(t, yid, t(na+1:end), Ysim_id(na+1:end))
title("Identification")
legend("Real", "Simulation")

figure,
plot(t, yval, t(na+1:end), Ysim_val(na+1:end))
title("Validation")
legend("Real", "Simulation")
```

Listing 7 – Finding the optimal na=nb and m (minimum MSE)

```
MSEId = zeros(6, 6);
MSEVal = zeros(6, 6);
MSEIdSIM = zeros(6, 6);
MSEValSIM = zeros(6, 6);

for n = 1:6
    for m = 1:6
        [Yid_pred, Yval_pred, Ysim_id, Ysim_val] = Ycalc(uid, yid, uval, yval, n, n, m);
        [MSEId(n,m), MSEVal(n,m), MSEIdSIM(n,m), MSEValSIM(n,m)] = MSEcompute(yid, yval, Yid_pred, Yval_pred, Ysim_id, Ysim_val, n);
    end
end

[MSEidOpt, idxId] = min(MSEId);
[MSEisOpt, idxIS] = min(MSEIdSIM);
[MSEvalOpt, idxV] = min(MSEVal);
[MSEvsOpt, idxVS] = min(MSEValSIM);

[msevalOpt, mval] = min(MSEvalOpt);
mseidOpt = MSEidOpt(mval);
[msevsOpt, mvs] = min(MSEvsOpt);
mseisOpt = MSEisOpt(mvs);

disp(['MSE for Predicted Validation data: ', num2str(msevalOpt), ' for na=nb=', num2str(idxV(mval)), ' and m=', num2str(mval)]);
disp(['MSE for Predicted Identification data: ', num2str(mseidOpt), ' for na=nb=', num2str(idxV(mval)), ' and m=', num2str(mval)]);
disp(['MSE for Simulated Validation data: ', num2str(msevsOpt), ' for na=nb=', num2str(idxVS(mvs)), ' and m=', num2str(mvs)]);
disp(['MSE for Simulated Identification data: ', num2str(mseisOpt), ' for na=nb=', num2str(idxIS(mvs)), ' and m=', num2str(mvs)]);

np = idxV(mval);
ns = idxVS(mvs);
```

Code Figures

Listing 8 – Plotting results for optimal model orders and polynomial degree

```
[Yid_pred, Yval_pred, ~, ~] = Ycalc(uid, yid, uval, yval, np, np, mval);
figure,
plot(t,yid, t(np+1:end), Yid_pred(np+1:end))
title("Identification for optimal parameters")
legend("Real", "Predicted")

figure,
plot(t, yval, t(np+1:end), Yval_pred(np+1:end))
title("Validation for optimal parameters")
legend("Real", "Predicted")

[~, ~, Ysim_id, Ysim_val] = Ycalc(uid, yid, uval, yval, ns, ns, mvs);
figure,
plot(t, yid, t(ns+1:end), Ysim_id(ns+1:end))
title("Identification for optimal parameters")
legend("Real", "Simulated")

figure,
plot(t, yval, t(ns+1:end), Ysim_val(ns+1:end))
title("Validation for optimal parameters")
legend("Real", "Simulation")
```

Listing 9 – Comparing the models

```
id = iddata(yid, uid, Ts);
val = iddata(yval(ns+1:end), uval(ns+1:end), Ts);
id_pred = iddata(Yid_pred, uid, Ts);
val_pred = iddata(Yval_pred(ns+1:end), uval(ns+1:end), Ts);
id_sim = iddata(Ysim_id, uid, Ts);
val_sim = iddata(Ysim_val(ns+1:end), uval(ns+1:end), Ts);

figure,
compare(id, id_pred);
figure,
compare(val, val_pred);

figure,
compare(id, id_sim);
figure,
compare(val, val_sim);
```